OOP/DB Project

Hideg Mark-Alex, 30421

1. Project Overview

The Hospital Management System (HMS) is a comprehensive web-based application designed to digitize the administration of hospital data. It serves as a central "Electronic Health Record" (EHR) system, enabling administrators and medical staff to manage patient admissions, infrastructure (wards/rooms), staff assignments, and complex medical histories including diagnoses and prescriptions.

The core philosophy of the application is the **"Patient 360"** view, which aggregates disparate data sources (billing, labs, history, demographics) into a single, interactive dashboard for medical professionals.

2. Technology Stack

The project utilizes a standard multi-tier architecture, separating data persistence, business logic, and presentation layers.

- **Database (Persistence Layer):**
  - **PostgreSQL:** selected for its reliability and strict relational integrity constraints.
- **Backend (Business Logic Layer):**
  - **Language:** Java (JDK 25).
  - **Framework:** Spring Boot 3+ (utilizing Spring Web and Spring Data JPA).
  - **Build Tool:** Maven.
  - **OOP Utilities:** Lombok (for encapsulation boilerplate), Jackson (for JSON serialization).
- **Frontend (Presentation Layer):**
  - **Structure:** HTML5.
  - **Styling:** Bootstrap 4.5 (Responsive CSS framework).
  - **Logic:** Vanilla JavaScript (ES6+) utilizing asynchronous `fetch` API for REST communication.

3. Object-Oriented Design (OOP) Analysis

The backend is built upon Object-Oriented principles. The domain model maps real-world hospital entities to Java classes.

3.1. Class Structure & Relationships

The system relies heavily on **Composition** and **Association** to link entities.

- **`Patient` Class:**
  - *Role:* The central entity of the system.
  - *Encapsulation:* All fields (`firstName`, `email`, etc.) are private and accessed via Getters/Setters (generated by Lombok `@Data`).
  - *Associations:*
    - **Has-a** `Doctor` (Many-to-One).
    - **Has-a** `Ward` and `Room` (Many-to-One).
- **`MedicalRecord` Class:**
  - *Role:* Represents a single diagnosis event.
  - *Composition:* A record *cannot exist* without a Patient and a Doctor. It is strictly composed of these two entities alongside the medical data (`diagnosis`, `treatmentPlan`).
- **`Prescription` & `PrescriptionItem` (One-to-Many):**
  - *Role:* Handles medication orders.
  - *Relationship:* A generic `Prescription` object (the "header" containing date and doctor) maintains a `List<PrescriptionItem>`. This demonstrates the **One-to-Many** relationship in OOP, where a parent object manages a collection of child objects.

3.2. OOP Principles Applied

1. **Encapsulation:** Data hiding is enforced through private fields and public accessors. API Controllers act as the public interface for these internal models.
2. **Abstraction:** The `Repository` layer (Spring Data JPA) abstracts away the complexity of raw SQL queries, allowing the business logic to interact with high-level Java objects instead of database rows.
3. **Polymorphism:** The API endpoints handle different HTTP methods (`GET`, `POST`) on the same URL path (e.g., `/api/patients`), behaving differently based on the request context (fetching vs. updating).

4. Database Design (Relational Model)

The database schema reflects the OOP structure but adheres to normalization rules (3NF).

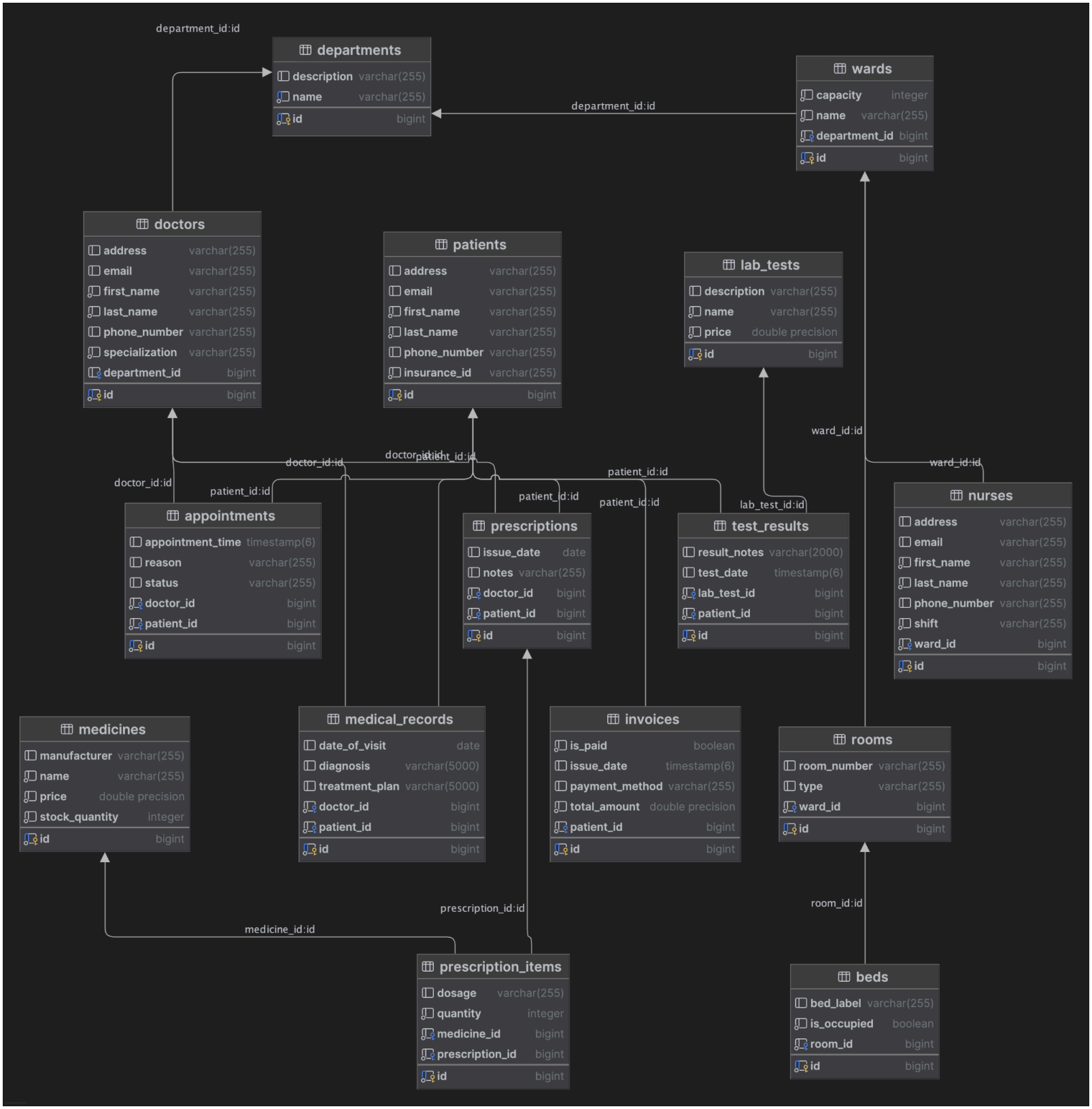
4.1. Entity-Relationship Diagram (ERD) Overview

- **`patients` Table:**
  - Primary Key: `id` (BigInt).
  - Foreign Keys: `ward_id`, `room_id`, `doctor_id`, `nurse_id`.
- **`medical_records` Table:**
  - Foreign Keys: Links to `patients(id)` and `doctors(id)`.
  - Constraint: Ensures history remains even if a patient is discharged.
- **`prescriptions` Table:**
  - Acts as the parent table for medication orders.
  - Columns: `issue_date` (Date), `notes` (Text).
- **`prescription_items` Table:**
  - The child table for specific drugs.
  - Foreign Key: `prescription_id` (Links back to parent).
  - Foreign Key: `medicine_id` (Links to the catalog of drugs).

4.2. Data Integrity

- **Foreign Key Constraints:** The database prevents the deletion of a Doctor if they are currently assigned to patients or prescriptions (Referential Integrity).
- **Date Formats:** `LocalDate` in Java is mapped to SQL `DATE` (YYYY-MM-DD), ensuring prescriptions are not tied to specific timestamps, whereas Admissions might use `TIMESTAMP`.

Diagram


department_id:id
departments
description varchar(255)
name varchar(255)
id bigint
department_id:id
wards
capacity integer
name varchar(255)
department_id bigint
id bigint
doctors
address varchar(255)
email varchar(255)
first_name varchar(255)
last_name varchar(255)
phone_number varchar(255)
specialization varchar(255)
department_id bigint
id bigint
patients
address varchar(255)
email varchar(255)
first_name varchar(255)
last_name varchar(255)
phone_number varchar(255)
insurance_id varchar(255)
id bigint
lab_tests
description varchar(255)
name varchar(255)
price double precision
id bigint
ward_id:id
ward_id:id
doctor_id:id
doctor_id:id
patient_id:id
patient_id:id
patient_id:id
patient_id:id
patient_id:id
lab_test_id:id
appointments
appointment_time timestamp(6)
reason varchar(255)
status varchar(255)
doctor_id bigint
patient_id bigint
id bigint
prescriptions
issue_date date
notes varchar(255)
doctor_id bigint
patient_id bigint
id bigint
test_results
result_notes varchar(2000)
test_date timestamp(6)
lab_test_id bigint
patient_id bigint
id bigint
nurses
address varchar(255)
email varchar(255)
first_name varchar(255)
last_name varchar(255)
phone_number varchar(255)
shift varchar(255)
ward_id bigint
id bigint
medicines
manufacturer varchar(255)
name varchar(255)
price double precision
stock_quantity integer
id bigint
medical_records
date_of_visit date
diagnosis varchar(5000)
treatment_plan varchar(5000)
doctor_id bigint
patient_id bigint
id bigint
invoices
is_paid boolean
issue_date timestamp(6)
payment_method varchar(255)
total_amount double precision
patient_id bigint
id bigint
rooms
room_number varchar(255)
type varchar(255)
ward_id bigint
id bigint
prescription_id:id
room_id:id
medicine_id:id
prescription_items
dosage varchar(255)
quantity integer
medicine_id bigint
prescription_id bigint
id bigint
beds
bed_label varchar(255)
is_occupied boolean
room_id bigint
id bigint

5. User Manual & Functional Behavior

The Hospital Management System is divided into three core functional modules: Patient Administration, Staff Management, and Infrastructure Control. Below is a guide on how to utilize these features.

5.1. The Dashboard & Room Occupancy

The main dashboard serves as the command center for hospital logistics.

- **Ward Overview:** The dashboard displays a list of all hospital Wards (e.g., "Cardiology", "ICU", "Pediatrics").
- **Real-Time Occupancy:** Clicking on a Ward reveals the rooms within it. The system visually indicates capacity:
    - **Green:** Room has available beds.
    - **Red:** Room is fully occupied.
    - *Behavior:* The system enforces business logic that prevents an administrator from assigning a new patient to a "Red" (full) room during the admission process. This prevents overbooking and ensures resource availability.

5.2. Staff Management (Doctors & Nurses)

Before patients can be treated, medical staff must be registered in the system to appear in assignment dropdowns.

- **Onboarding Doctors:**
    1. Navigate to the **"Staff Directory"**.
    2. Click **"Add Doctor"**.
    3. **Required Fields:** Enter First Name, Last Name, and select a **Department** (e.g., Orthopedics).
    4. **Outcome:** The Doctor is saved to the database and immediately becomes available for selection in the "Patient Profile" for primary care assignment and prescription authorization.
- **Registering Nurses:**
    1. Click **"Add Nurse"** in the Staff Directory.
    2. Enter personal details and Ward assignment.
    3. **Outcome:** The nurse is linked to a specific Ward, allowing the system to suggest them automatically when a patient is admitted to that specific area.

5.3. Patient 360 Profile

Clicking "View Profile" on any patient in the list loads the detailed view, aggregating all data relationships.

- **Infrastructure Status:** Shows exactly where the patient is located (Ward Name & Room Number).
- **Medical History:** A chronological timeline of past diagnoses.
- **Financials:** Automatically aggregates all unpaid invoices to show a "Total Due" balance.

5.4. Dynamic Editing (Admissions & Transfers)

The system features a "Single Page Application" feel using JavaScript Modals to handle patient movement.

1. **Click "Edit Profile":** A popup loads the patient's current data.
2. **Infrastructure Logic:** When an administrator changes the **"Ward"** dropdown, the system triggers an event to filter the **"Room"** dropdown. It scans the database and only displays rooms that belong to the selected Ward and have **Capacity < Max Capacity**.
3. **Staff Reassignment:** Changing the Primary Doctor or Nurse updates the foreign key links immediately upon saving.

5.5. Clinical Workflow: Prescribing Medication

Doctors use the profile to issue new prescriptions without leaving the patient's chart.

1. **Add Medicine:** The user clicks the **"+ Add Med"** button to generate a new input row.
2. **Catalog Selection:** The "Medicine" dropdown pulls live data from the `medicines` table (Inventory), ensuring doctors can only prescribe drugs that exist in the system.
3. **Save Validation:**
    - The system checks if a Doctor is selected.
    - It ensures the Date format is strictly adhered to (YYYY-MM-DD).
    - It strips unnecessary timestamps to ensure compatibility with the database `DATE` type.
    - Upon success, the prescription is saved, and the "Active Prescriptions" list updates instantly without a page reload.

6. Conclusion

This project demonstrates the successful integration of Object-Oriented Programming logic with Relational Database storage. By leveraging the Spring Boot ecosystem, the application achieves a clean separation of concerns, ensuring that the complex relationships between Patients, Doctors, and Medical Records are handled robustly and efficiently.